Trabajo Integrador - Entrega 2

Plataforma de competencias de robótica y desafíos técnicos

Introducción

El trabajo integrador se desarrolla en grupos de cinco integrantes sobre esta consigna. La complejidad de negocio es suficiente para que la calidad del modelado, la separación de responsabilidades y la extensibilidad resulten determinantes: no alcanza con que el sistema funcione.

Enunciado

Una organización realiza competencias de robótica con categorías, desafíos, rondas y sistemas de puntuación diferentes. Algunos desafíos se puntúan por tiempo, otros por objetivos cumplidos, precisión, consumo de recursos, evaluación de jueces o una combinación de factores.

Cada edición publica un reglamento. Los resultados deben poder recalcularse utilizando exactamente la versión de reglas correspondiente. Además, el sistema debe explicar cómo se obtuvo cada puntaje y conservar ajustes, penalizaciones y decisiones sobre apelaciones.

El equipo deberá desarrollar una aplicación que permita configurar competencias, registrar equipos, programar rondas, capturar resultados, calcular puntajes, resolver empates, publicar posiciones y gestionar reclamos.

Requisitos funcionales obligatorios

Capacidad	Comportamiento esperado
Configuración del evento	Crear temporada, competencia, categorías y fechas
Registro de equipos	Administrar integrantes, robot, categoría y documentación
Elegibilidad	Validar restricciones de edad, composición, robot o categoría
Configuración de desafíos	Definir métricas, reglas de puntaje, penalizaciones y bonificaciones
Programación	Crear rondas, turnos, pistas y asignaciones de jueces
Captura de resultados	Registrar tiempos, mediciones, objetivos y evaluaciones
Cálculo explicable	Obtener puntaje total y detalle de cada contribución
Ranking	Ordenar equipos según puntaje y reglas de desempate
Publicación	Diferenciar resultados provisionales y definitivos
Apelaciones	Presentar, revisar, aceptar o rechazar reclamos
Recálculo	Reprocesar posiciones después de una corrección
Auditoría	Conservar los valores originales y todas las modificaciones

Funcionalidades nuevas de esta entrega

Además de la implementación completa de los requisitos funcionales anteriores y de los requisitos no funcionales, esta entrega incorpora las funcionalidades descritas a continuación.

F1 - Mejores N de M rondas

Algunas competencias pasan a contabilizar solo los mejores resultados de cada equipo: por ejemplo, las mejores tres de cinco rondas. El reglamento define N y M por desafío.

Criterios de aceptación: un desafío puede configurarse para contabilizar las mejores N de M rondas; la explicación del puntaje indica qué rondas fueron consideradas y cuáles descartadas; el cambio no exige modificar las reglas de puntaje existentes.

F2 - Límite global a la suma de bonificaciones

El reglamento incorpora un tope: la suma de todas las bonificaciones de un desafío no puede superar un máximo configurable, sin importar cuántas bonificaciones individuales se hayan obtenido.

Criterios de aceptación: el tope se aplica sobre el conjunto de bonificaciones y no sobre cada una; la explicación del puntaje muestra las bonificaciones obtenidas, el tope aplicado y el recorte resultante; agregar el tope no exige reescribir las reglas de bonificación existentes.

F3 - Desafío mixto: mediciones automáticas y jueces

Se incorpora un desafío cuyo puntaje combina mediciones capturadas automáticamente (tiempo, sensores) con evaluaciones de un panel de jueces. Ambas fuentes llegan por separado y el puntaje solo puede calcularse cuando las dos están completas.

Criterios de aceptación: el sistema acepta resultados de dos fuentes distintas para el mismo turno; el puntaje queda pendiente hasta que ambas fuentes estén completas y ese estado es visible; la fórmula combina ambas contribuciones y la explicación las detalla por separado.

Requisitos no funcionales

Arquitectura

La arquitectura del proyecto debe seguir una de las vistas en clase: Clean Architecture, Domain Driven Design o Hexagonal.

El backend debe exponer una **API REST**. El frontend debe contener lo mínimo indispensable para llevar a cabo las funcionalidades pedidas.

Testing e integración continua

Deben existir pruebas unitarias, de repositorio y de API. Los mocks sustituyen colaboradores en pruebas unitarias; no reemplazan las pruebas reales de persistencia ni de API.

La integración continua debe compilar, ejecutar las pruebas y bloquear la integración de pull requests que fallen.

Entrega 2 - Aplicación completa

Objetivo: comprobar que la aplicación resuelve los principales casos de uso, que su arquitectura respeta límites claros y que el diseño tolera una evolución significativa: las funcionalidades nuevas descritas en este documento.

El entregable central es la **aplicación completa**: un proyecto ejecutable que implementa todos los requisitos funcionales, incluidas las funcionalidades nuevas, sobre el módulo de dominio, y que cumple los requisitos no funcionales descritos en este documento.

Entregables obligatorios

Entregable	Contenido mínimo
Repositorio	URL del repositorio de GitHub y hash del último commit que corresponde a la entrega
Código de la aplicación	Módulo de dominio, casos de uso, infraestructura (persistencia, integraciones, procesos) y API REST, organizados según la arquitectura elegida
Funcionalidades nuevas	F1, F2 y F3 implementadas, probadas y documentadas
Tests	Unitarios, parametrizados, de repositorio y de API; deben ejecutarse en la integración continua
Datos de demostración	Script, migración o fixture repetible que permita reproducir la demostración sugerida
Documentación	README ejecutable con instrucciones para compilar, ejecutar y probar, y contratos de la API
Decisiones de diseño	Archivo `DESIGN.md` en la raíz del repositorio que enumera las decisiones de diseño principales: qué patrón o principio se aplicó, dónde y por qué; qué alternativas se descartaron; y qué patrones se decidió no aplicar, con sus consecuencias. Por cada funcionalidad nueva incluye además las clases agregadas y modificadas, las refactorizaciones realizadas y la deuda técnica que se decidió no resolver

Demostración sugerida

La demostración puede incluir al menos tres desafíos (uno mixto), diez tipos de reglas, una regla compuesta, una penalización, bonificaciones con tope global, un desafío con mejores N de M rondas, tres criterios encadenados de desempate y una apelación que provoque el recálculo del ranking.

Qué no se exige en esta entrega

No se exigen microservicios ni despliegue en la nube. El esfuerzo debe concentrarse en el backend, que es lo que se evalúa aquí.